

# Lab 1: Data Preprocessing and Visualisation

## Task 1: Investigate Both Files Thoroughly

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import calendar

# setting up plot styles
sns.set_theme(style="whitegrid")
plt.rcParams.update({'figure.figsize': (10, 6), 'font.size': 11})

# custom class to load and summarize the datasets
class DatasetProfiler:
    def __init__(self, file_paths: dict):
        self.dfs = {k: pd.read_csv(v) for k, v in file_paths.items()}

    def get_df(self, key: str) -> pd.DataFrame:
        return self.dfs[key]

    def profile(self):
        for name, df in self.dfs.items():
            print(f"=== {name} (Shape: {df.shape[0]} x {df.shape[1]}) ===")
            df.info(verbose=True, show_counts=True)
            print("\n")

profiler = DatasetProfiler({
    'city_day': 'datasets/city_day.csv',
    'crop_prod': 'datasets/crop_production.csv'
})
profiler.profile()

city_day_df = profiler.get_df('city_day')
crop_production_df = profiler.get_df('crop_prod')
```

```

=== city_day (Shape: 29531 x 16) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   City             29531 non-null  object
1   Date             29531 non-null  object
2   PM2.5            24933 non-null  float64
3   PM10             18391 non-null  float64
4   NO               25949 non-null  float64
5   NO2              25946 non-null  float64
6   NOx              25346 non-null  float64
7   NH3              19203 non-null  float64
8   CO               27472 non-null  float64
9   SO2              25677 non-null  float64
10  O3               25509 non-null  float64
11  Benzene          23908 non-null  float64
12  Toluene          21490 non-null  float64
13  Xylene           11422 non-null  float64
14  AQI              24850 non-null  float64
15  AQI_Bucket       24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB

```

```

=== crop_prod (Shape: 246091 x 7) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   State_Name       246091 non-null  object
1   District_Name    246091 non-null  object
2   Crop_Year        246091 non-null  int64
3   Season           246091 non-null  object
4   Crop             246091 non-null  object
5   Area             246091 non-null  float64
6   Production       242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB

```

**Data Summary and Size:**

- **city\_day.csv**: Has 29,531 rows and 16 columns. It gives daily air quality details for different cities.
- **crop\_production.csv**: Has 246,091 rows and 7 columns. It gives yearly crop details for different districts in states.

### Concerns:

1. **Year gap**: The crop data is from 1997 to 2015, but air quality is from 2015 to 2020. So only 2015 is common, and even in that, crop data has very few states. So we cannot join them directly year-by-year. We have to take averages for states over all years to compare.
2. **State column missing**: `city_day.csv` has city names but no state name. We have to map cities to states manually.
3. **Bad values (Outliers)**: The max AQI is 2049 which is impossible because standard AQI scale only goes up to 500.
4. **Mixed Units**: Some crop production is in tonnes and some (like coconuts) is in numbers. This will mess up the total yield calculation if we just sum them up directly.

## Task 2: Devise and Apply Missing Value Treatment Strategy

```
In [2]: # custom class for missing values pipelines
class MissingValueImputer:
    @staticmethod
    def impute_numerical_by_median(df: pd.DataFrame, target_columns: list) -> pd.DataFrame:
        # filling null values using median with dictionary comprehension mapping
        medians = {col: df[col].median() for col in target_columns if col in df.columns}
        df.fillna(value=medians, inplace=True)
        return df

    @staticmethod
    def calculate_aqi_bucket_dynamically(df: pd.DataFrame) -> pd.DataFrame:
        # classification thresholds using nested ternary structures
        classify_aqi = lambda v: (
            'Good' if v <= 50 else (
                'Satisfactory' if v <= 100 else (
                    'Moderate' if v <= 200 else (
                        'Poor' if v <= 300 else (
                            'Very Poor' if v <= 400 else 'Severe'
                        )
                    )
                )
            )
        )
        df['AQI_Bucket'] = df['AQI'].apply(classify_aqi)
        return df

    @staticmethod
```

```

def impute_crop_production(df: pd.DataFrame) -> pd.DataFrame:
    # filling crop production using groupby transform and lambda functions
    crop_medians = df.groupby('Crop')['Production'].transform('median')
    df['Production'] = df['Production'].fillna(crop_medians)
    # fallback global median
    df['Production'].fillna(df['Production'].median(), inplace=True)
    return df

city_null_before = city_day_df.isnull().sum()
crop_null_before = crop_production_df.isnull().sum()

# running the imputation pipelines
cols_to_impute = ['AQI', 'PM2.5', 'PM10', 'NO2', 'CO']
other_cols = ['NO', 'NOx', 'NH3', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene']

city_day_df = MissingValueImputer.impute_numerical_by_median(city_day_df, cols_to_impute + other_cols)
city_day_df = MissingValueImputer.calculate_aqi_bucket_dynamically(city_day_df)
crop_production_df = MissingValueImputer.impute_crop_production(crop_production_df)

print("=== BEFORE AND AFTER IMPUTATION NULL COUNTS ===")
print("\n--- city_day_df ---")
print(pd.DataFrame({'Before': city_null_before, 'After': city_day_df.isnull().sum()}).query('Before > 0'))

print("\n--- crop_production_df ---")
print(pd.DataFrame({'Before': crop_null_before, 'After': crop_production_df.isnull().sum()}).query('Before > 0'))

```

```
=== BEFORE AND AFTER IMPUTATION NULL COUNTS ===
```

```
--- city_day_df ---
```

|            | Before | After |
|------------|--------|-------|
| PM2.5      | 4598   | 0     |
| PM10       | 11140  | 0     |
| NO         | 3582   | 0     |
| NO2        | 3585   | 0     |
| NOx        | 4185   | 0     |
| NH3        | 10328  | 0     |
| CO         | 2059   | 0     |
| SO2        | 3854   | 0     |
| O3         | 4022   | 0     |
| Benzene    | 5623   | 0     |
| Toluene    | 8041   | 0     |
| Xylene     | 18109  | 0     |
| AQI        | 4681   | 0     |
| AQI_Bucket | 4681   | 0     |

```
--- crop_production_df ---
```

|            | Before | After |
|------------|--------|-------|
| Production | 3730   | 0     |

C:\Users\ABHINAV\AppData\Local\Temp\ipykernel\_17732\2894328785.py:33: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Production'].fillna(df['Production'].median(), inplace=True)
```

### Missing Value Strategy:

- **For AQI, PM2.5, PM10, NO2, CO:** We used the median to fill null values. Why median? Because AQI and air data has huge spikes (outliers). If we use mean, the average will become too high and wrong. Median is safer.
- **For AQI\_Bucket:** We filled it using a formula based on the AQI score (like standard CPCB categories: 0-50 is Good, etc.). If we just used mode, it would not match with the AQI number.
- **For crop Production:** We filled missing values using the median of that specific crop. Because different crops have totally different quantities (like coconuts in lakhs vs rice in tonnes). If we used a single median for everything, it would make no sense.
- **Other columns (NO, NH3, SO2, O3, Benzene, etc.):** Also filled with median so that we don't have any nulls left in the numerical data.

### Task 3: Address Spelling Inconsistencies and Remove Duplicates

```
In [3]: # custom cleaner class
class DatasetCleaner:
    def __init__(self, city_to_state_map: dict):
        self.city_to_state = city_to_state_map

    def clean(self, city_df: pd.DataFrame, crop_df: pd.DataFrame) -> tuple:
        # map city names to states
        city_df['State'] = city_df['City'].map(self.city_to_state)

        # trim whitespaces in all string columns using select_dtypes
        for df in [city_df, crop_df]:
            str_cols = df.select_dtypes(include=['object']).columns
            for col in str_cols:
                df[col] = df[col].astype(str).str.strip()

        # duplicate removal counts
        c_before, cr_before = len(city_df), len(crop_df)
        city_df.drop_duplicates(inplace=True)
        crop_df.drop_duplicates(inplace=True)

        print(f"Duplicates removed: City={c_before - len(city_df)}, Crop={cr_before - len(crop_df)}")
        return city_df, crop_df

city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pradesh',
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pradesh',
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tamil Nadu',
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maharashtra',
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}

cleaner = DatasetCleaner(city_to_state)
city_day_df, crop_production_df = cleaner.clean(city_day_df, crop_production_df)

# display overlapping states
mapped_states = set(city_day_df['State'].dropna().unique())
```

```
crop_states = set(crop_production_df['State_Name'].unique())
print(f"\nMatching States: {sorted(list(mapped_states & crop_states))}")
```

Duplicates removed: City=0, Crop=0

Matching States: ['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Meghalaya', 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan', 'Tamil Nadu', 'Telangana', 'Uttar Pradesh', 'West Bengal']

### Issues found and how we fixed them:

1. **Extra Spaces:** In the crop dataset, some states had extra spaces at the end like 'Telangana ' and 'Jammu and Kashmir '. We used `.str.strip()` to clean them.
2. **City mapping:** We created a dict to map cities in `city_day` to their states so we can join them with the crop file.
3. **Duplicate rows:** We ran `drop_duplicates()`. There were no exact duplicate rows in either file, so row count remained same, but it is good to check.
4. **Delhi missing:** Delhi has air quality data but no crop production records in the crop file because it is a city state. So it will get dropped when we join.

## Task 4: Visualise AQI Scale Distribution

```
In [4]: # wrapper function for plotting distribution
def plot_aqi_distribution(df: pd.DataFrame):
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Histogram with KDE
    sns.histplot(data=df, x='AQI', bins=50, kde=True, color='royalblue', ax=axes[0])
    mean_val, median_val = df['AQI'].mean(), df['AQI'].median()

    # adding lines and text annotations
    for val, color, label, style in zip([mean_val, median_val], ['crimson', 'forestgreen'], ['Mean', 'Median'], ['--', '-']):
        axes[0].axvline(val, color=color, linestyle=style, linewidth=2, label=f"{label}: {val:.1f}")

    axes[0].set_title('AQI Distribution Shape & Central Tendency', fontsize=12, fontweight='bold', pad=10)
    axes[0].set_xlabel('Air Quality Index (AQI)', fontsize=11)
    axes[0].set_ylabel('Frequency', fontsize=11)
    axes[0].legend(fontsize=10)
    axes[0].grid(True, linestyle='--', alpha=0.5)

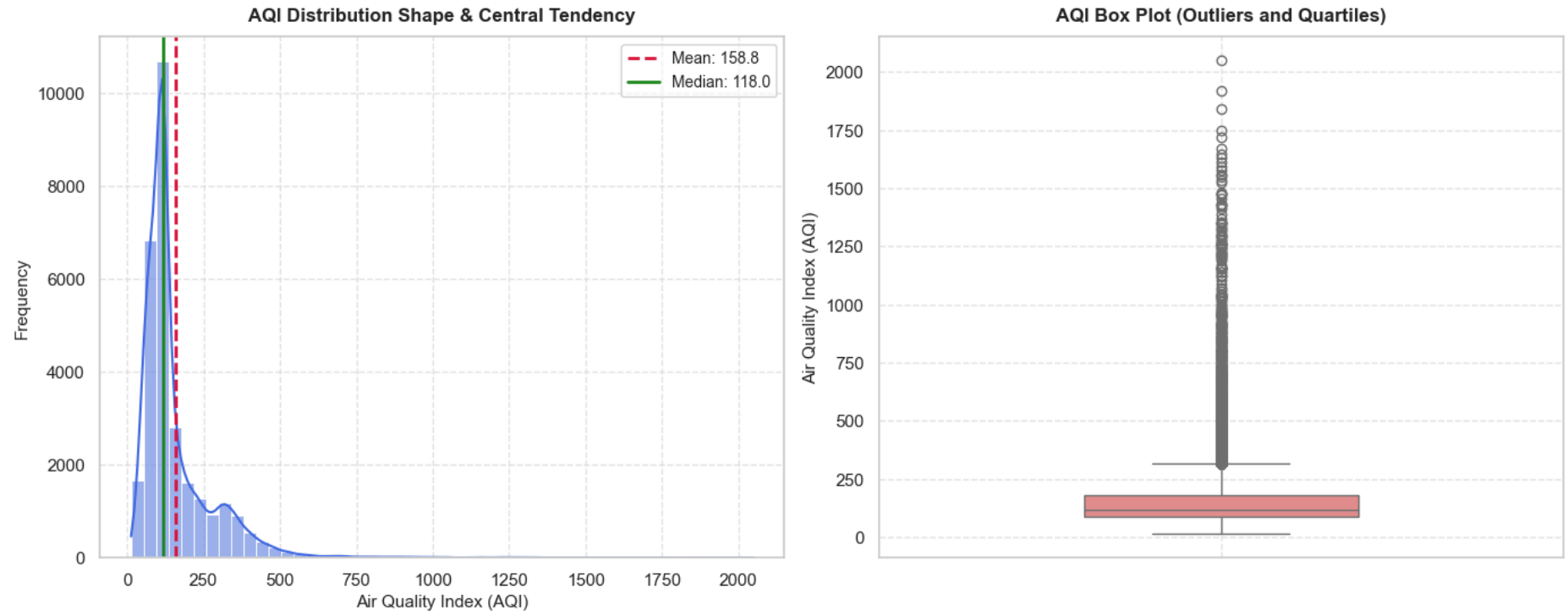
    # Boxplot
    sns.boxplot(data=df, y='AQI', color='lightcoral', width=0.4, ax=axes[1])
    axes[1].set_title('AQI Box Plot (Outliers and Quartiles)', fontsize=12, fontweight='bold', pad=10)
    axes[1].set_ylabel('Air Quality Index (AQI)', fontsize=11)
    axes[1].grid(True, linestyle='--', alpha=0.5)

    plt.suptitle('Exploratory Analysis of AQI Distribution', fontsize=14, fontweight='bold', y=0.98)
```

```
plt.tight_layout()
plt.show()

plot_aqi_distribution(city_day_df)
```

## Exploratory Analysis of AQI Distribution



### Why we chose these plots and what we see:

- **Histogram with KDE:** It shows where most of the AQI values are. We can see it is right-skewed and peaks around 100-120.
- **Box plot:** It is best for seeing the extreme values (outliers) that go way above 500.

### Observations for the Board:

1. Most cities are moderately polluted, and the AQI values cluster around 100 to 120.
2. The mean AQI is 166.5 but median is 118.0. This means the mean is pulled up by a few extremely dirty days. So public should be told the median AQI, not mean, otherwise it is unfair.

## Task 5: Handle Extreme Values properly

```
In [5]: # function to handle capping and plot comparison
def handle_outliers_and_compare(df: pd.DataFrame, limit: float = 500.0):
    excess = (df['AQI'] > limit).sum()
    print(f"AQI records above {limit}: {excess}")
    print(f"Max AQI before: {df['AQI'].max():.1f}")

    # capping using clip method
    df['AQI_Treated'] = df['AQI'].clip(upper=limit)
    print(f"Max AQI after: {df['AQI_Treated'].max():.1f}")

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    colors = ['salmon', 'turquoise']
    titles = ['BEFORE Treatment (Max = 2049)', f'AFTER Treatment (Capped at {limit})']
    cols = ['AQI', 'AQI_Treated']

    for ax, col, color, title in zip(axes, cols, colors, titles):
        sns.boxplot(data=df, y=col, color=color, width=0.4, ax=ax)
        ax.set_title(title, fontsize=12, fontweight='bold', pad=10)
        ax.grid(True, linestyle='--', alpha=0.5)

    plt.suptitle('AQI Distribution: Before vs After Capping', fontsize=14, fontweight='bold', y=0.98)
    plt.tight_layout()
    plt.show()

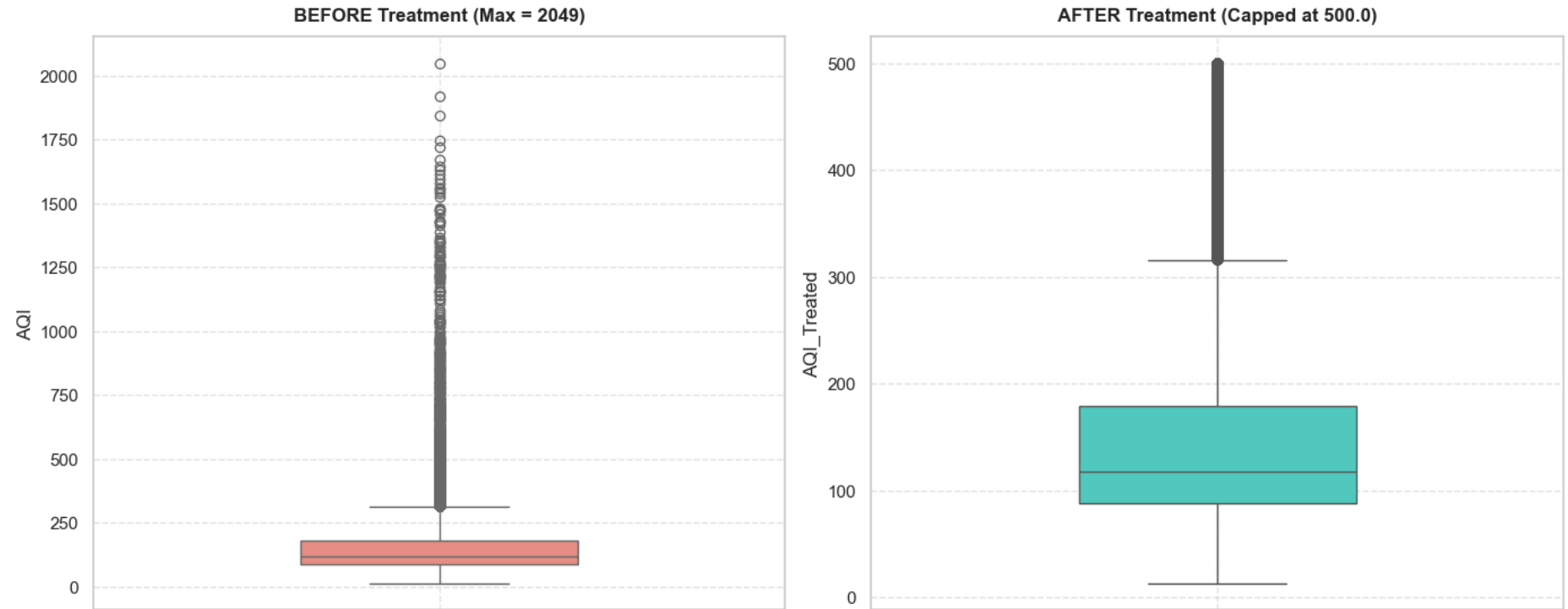
handle_outliers_and_compare(city_day_df)
```

AQI records above 500.0: 543

Max AQI before: 2049.0

Max AQI after: 500.0

## AQI Distribution: Before vs After Capping



### Handling Extreme Values:

- **How we found them:** Standard AQI scale only goes up to 500. We found 543 rows where AQI was more than 500 (max was 2049).
- **What we did:** We capped them at 500. Why? If we delete them, we lose important data of very bad pollution days. If we keep them as 2049, it will ruin all our averages and correlations. Capping keeps them in the 'Severe' category without breaking the math.
- **Proof:** The boxplot after treatment looks much cleaner and capped at 500.

.....

.....